Structured Data Extraction from Bilateral Contracts: A Stage-wise Evaluation of an End-to-End Retrieval-Augmented Pipeline

ContractExtraction project

Code fair submission and progress report, 28 October 2026. Development-split results; test-split results pending.

Abstract

Two-party commercial contracts carry the facts a business must act on, such as who the parties are, what is paid and delivered, which penalties apply and how risk is shared, yet they are written for people and stored in formats built for page layout rather than for machines. Getting those facts out reliably depends on every link of a chain: turning the document into text, finding the right passages, reading them with a language model, and checking the result. We describe an end-to-end platform that implements this chain for contracts in PDF, scanned, HTML, Word and plain-text form, and we evaluate it one stage at a time. Documents are converted page by page into Markdown with PyMuPDF, PyMuPDF4LLM or OCR, chunked and indexed in Qdrant with dense and BM25 keyword vectors; each field is retrieved with expanded queries, optional keyword fusion, cover- and signature-page anchors and an optional cross-encoder reranker, and extracted by a prompted LLM. Because every stage is configuration, one component can be changed while the model, retriever and evaluator stay fixed, and each comparison is reported as a paired difference with a 95% confidence interval over documents. On the development split (36 of 144 public contracts, 18 contract types), retrieval is the main lever: the current system, seven retrieval configurations with Llama 3.1 8B and an LLM judge, reaches a semantic score of 0.31, while a single hybrid configuration with anchor chunks that passes 10 or 20 chunks to Qwen3 30B reaches 0.57–0.59 at about one US cent per contract. Neither the judge nor the reranker improves the answers, and more context brings more values for fields the contract does not contain, which an explicit prompt rule does not reduce. Comparing document representations and fine-tuning a smaller extraction model are scheduled for the end of the programme, once the best retrieval and reading setup is fixed, so that fine-tuning starts from the strongest pipeline. The findings are meant to guide teams building contract-intelligence systems.

Keywords: contract analysis, information extraction, retrieval-augmented generation, hybrid retrieval, reranking, fine-tuning, LLM evaluation

1 Introduction

An organisation with hundreds of agreements needs to know, for each one, when it starts and ends, how it renews, what it costs and how either side can leave it. Reading these facts out by hand is slow and error-prone, so automatic extraction into a fixed schema of fields is an attractive target for LLMs. Contracts are long (median 16 pages in our corpus, up to 428), so the usual architecture splits each document into chunks, retrieves the chunks most relevant to each field, and asks the model to extract the value from them (retrieval-augmented generation, RAG).

Every stage of such a pipeline has free parameters: how text is chunked, which embedding model and similarity threshold select chunks, whether lexical search is fused with dense search, how many chunks reach the model, whether a reranker reorders them, which model reads them, and whether a second model judges the answer. Practitioner systems, including the one studied here, often combine several retrieval configurations and an LLM judge in the hope that the ensemble is more robust than any single setting. Whether these choices help is rarely measured, and when it is, retrieval and reading errors are usually mixed in a single end-to-end accuracy figure.

This report asks a practical question: **which components of a contract-extraction pipeline improve extraction quality, and by how much?** We answer it with controlled experiments that change one component

at a time, evaluate retrieval and extraction separately, and report paired differences with uncertainty. The work is ongoing; this progress report covers the baseline, retrieval, reranking, context-size and first prompt experiments on the development split, and the remaining experiments are planned in Section 9.

Scope and order of work. The original proposal also compared intermediate document representations and fine-tuned the extraction model. Both deliberately come after the retrieval and reader experiments, with fine-tuning last before the test run. The representation comparison (E7) is only meaningful under the best retrieval setup, and fine-tuning pays off only once the best retrieval and prompting approach is known: its training data are that pipeline's retrieved context, and a fine-tuned model must beat the strongest prompted baseline, not the current one. Both appear in the plan of Section 9 (Table 12).

The research questions addressed so far are:

RQ1 (baseline). How well does the current system perform, how does a single-configuration research baseline compare, and what is the smallest effect the data can resolve?

RQ2 (retrieval settings). Which retrieval settings decide whether the evidence for a field reaches the model?

RQ3 (reranking). Does a cross-encoder reranker improve evidence recall and, through it, the extracted values?

RQ4 (context size). How many retrieved chunks should the reader receive?

RQ5 (ensembling and judging). Does a multi-configuration ensemble with LLM-judge selection beat the best single configuration?

RQ6 (measurement). How should extracted values be scored when reference labels paraphrase the contract while models quote it?

RQ7 (invented values). Where do values for absent fields come from, and does an explicit instruction prevent them?

The contributions are a reproducible extraction platform that records every intermediate outcome (Section 3), a stage-wise evaluation protocol with paired statistics clustered by document (Section 5), the findings of Section 6, and an LLM-graded semantic score checked against a second, independent grader.

2 Related Work

2.1 Contract understanding

Legal NLP has produced several contract benchmarks. CUAD [6] annotates 41 clause categories in 510 commercial contracts as an extractive question-answering task, ContractNLI [8] frames contract review as document-level natural language inference over non-disclosure agreements, and LEDGAR [14] classifies provisions from SEC filings. These datasets label clause spans; our task differs in asking for normalised, typed values (ISO dates and durations, money with currency, party references, structured objects), which makes scoring harder and is one reason we introduce a semantic score.

2.2 Retrieval-augmented generation

RAG [9] conditions a generator on passages retrieved from an external corpus. Dense retrievers embed queries and passages in one vector space; lexical retrievers such as BM25 [12] match exact terms, which matters for legal text full of defined terms and numbers. Hybrid retrieval fuses both rankings, often with reciprocal rank fusion (RRF) [3]. Two-stage pipelines then reorder a larger candidate set with a cross-encoder reranker [11]; the BGE family [2] provides multilingual embedding and reranking models that run locally. Models do not use long contexts uniformly: information in the middle of a long prompt is used less reliably than information at its ends [10], which motivates the question of how many chunks to pass to the reader.

2.3 Evaluating RAG systems and LLM judges

RAGAS [4] and ARES [13] argue for evaluating retrieval and generation separately, using context relevance, faithfulness and answer quality. LLM-as-a-judge [16] uses a strong model to grade outputs and reports high agreement with humans on open-ended answers, but also position, verbosity and self-preference biases. We use an LLM grader only to score semantic equivalence against a reference label, and we measure its agreement with a second grader. For statistics we follow common NLP practice [5]: paired tests on the same items, bootstrap confidence intervals [1], McNemar's test [15] for paired binary outcomes, and Holm's correction [7] for multiple comparisons.

3 System

ContractExtraction is a research platform that takes a contract and returns a fixed set of fields, each with its value, the evidence chunks it was based on, and optionally a verdict from an LLM judge. A browser front end and a REST API sit in front of background workers; the workers claim queued runs from PostgreSQL, store vectors in Qdrant, and call either a local Ollama server or a hosted provider (OpenRouter). Every setting that shapes a run (models, prompts, chunking, retrieval configurations and fields) is configuration stored in the database, and each run stores the fully resolved configuration as a snapshot together with the git commit of the code that ran it. Table 1 summarises the stages.

Table 1. Pipeline stages.

Stage	Input → output	Main settings
Decompose	source file → page Markdown	converter per format (HTML, PDF via pymupdf / OCR / hybrid, DOCX, text); repeated header and footer removal
Index	pages → chunks → vectors	2,000-character chunks with 300 overlap, within a page; section heading prepended; nomic-embed-text embeddings and a BM25 keyword vector per chunk
Retrieve	field → ranked chunks	expansion queries per field, hits per query, top-k, score threshold, optional BM25 fused by RRF, optional anchor chunks, optional cross-encoder reranker
Extract	field prompt + chunks → value	one call per field (or per field group), JSON output in the field's typed format
Judge	value + evidence → verdict	separate LLM, PASS / FAIL; selects the best value across retrieval configurations

Retrieval. For each field the retriever issues the field label and its expansion queries, keeps a fixed number of hits per query, merges and deduplicates them, and passes the top k to the reader. A score threshold optionally drops low-similarity chunks. With keyword search on, the dense and BM25 rankings are fused by RRF. *Anchor chunks* add the first or last two chunks of the document for fields that are almost always on the cover page or in the preamble (title, parties, effective date) or on the signature page (signatories, execution date). The optional reranker scores a larger candidate pool (20 or 40 fused hits) with a cross-encoder, BAAI bge-reranker-v2-m3 quantised to int8 and run on CPU, and keeps the top k.

The current system. The default pipeline runs seven retrieval configurations, RAG01–RAG07 (Table 5), for every field, extracts a value under each with Llama 3.1 8B, and lets an LLM judge choose the best value. Its configurations were designed by hand and differ in several settings at once, so they serve here only as the baseline "current system". For the research grid we defined a clean baseline, **R-base** (dense retrieval, top 5, no threshold, query expansion, 5 hits per query), from which each arm changes a single setting.

3.1 Architecture and deployment

Figure 1 shows how the parts run. The browser talks only to the frontend, which reaches the backend through a gateway that adds request identifiers, timeouts, an upload size limit and an aggregated health check. The backend API never does pipeline work inside a request: it writes a queued run to PostgreSQL and returns. Workers claim queued runs with SELECT … FOR UPDATE SKIP LOCKED, send a heartbeat every 15

seconds, and write results back, so the API and the workers never call each other and any number of workers can run in parallel. Only the workers talk to Qdrant, the LLM providers and the reranker, which keeps the API fast. All pipeline configuration lives in the database, and each run executes from a frozen snapshot of it, so editing a configuration never changes a queued or finished run.

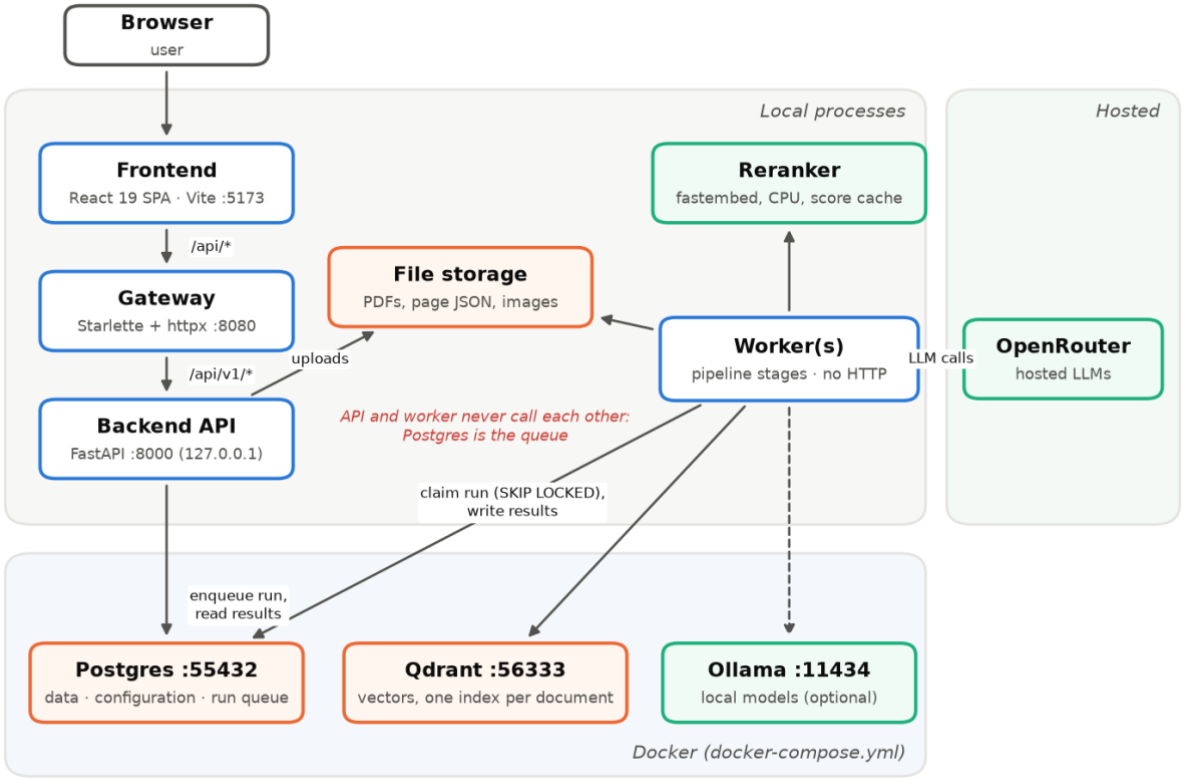


Figure 1. Deployment architecture. Blue: application processes; orange: data stores; green: model services. The API and the workers communicate only through the run queue in PostgreSQL.

3.2 Technology stack

Table 2. Technology stack.

Layer	Technologies
Frontend	React 19, TypeScript 5.9, Vite 8, Tailwind CSS 4, Radix UI, TanStack Query, React Router, openapi-fetch with types generated from the API's OpenAPI schema
Gateway	Python 3.13, Starlette, httpx (streaming reverse proxy), Uvicorn
Backend API and workers	Python 3.13, FastAPI, Pydantic 2, SQLAlchemy 2 with psycopg 3, Uvicorn; one package with API, worker and CLI entry points
Document processing	PyMuPDF and pymupdf4llm (PDF to Markdown, OCR and hybrid modes), BeautifulSoup and markdownify (HTML), mammoth (DOCX), Pillow (images)
Retrieval	Qdrant 1.12 (dense and BM25 sparse vectors per chunk, reciprocal rank fusion), Snowball stemmer, fastembed (ONNX cross-encoder reranker on CPU, SQLite score cache)
Models	Ollama (local Llama 3.1 8B, nomic-embed-text), OpenRouter (hosted Qwen3 30B, Llama 3.1 8B, Gemini, GPT-6 Luna and others) behind one OpenAI-compatible adapter
Data	PostgreSQL 16 (31 tables and 2 views: configuration, documents, runs and queue, results, ground truth), local file storage
Tooling	uv, npm, Docker Compose, pytest, Vitest with Testing Library and MSW, ESLint, TypeScript checks, one dev script (scripts/dev.sh) for start, status, smoke and test
Analysis and report	experiment CLI (create, run, export, compare, semantic), matplotlib, docx

3.3 User interface

The web interface covers the whole workflow: upload a contract in any supported format, follow a run's progress, inspect each extracted value next to its label and the retrieval configuration that produced it, read the extracted pages, and edit every configuration that shapes a run. Figure 2 shows the results of one run.

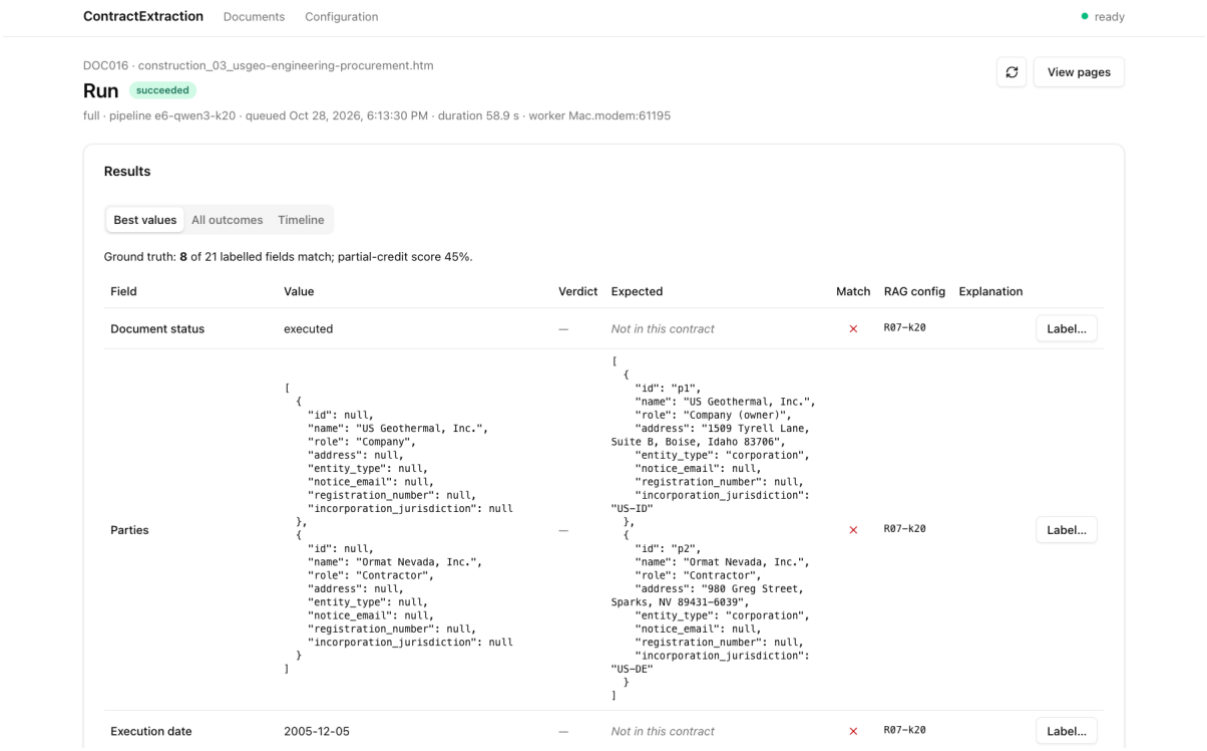


Figure 2. Results of a run (k = 20 arm): the extracted value, the judge's verdict, the expected label, whether they match and which retrieval configuration produced the value. The contract is a public SEC filing.

4 Data

4.1 Corpus

The corpus contains 144 publicly available contracts, most of them exhibits filed with the US SEC (EDGAR), covering 18 contract types (for example non-disclosure agreements, master services agreements, SaaS agreements, leases, loans, employment, licensing and share or asset purchase agreements) plus a fallback type. Documents come in several source formats, including HTML, text-layer PDF, scanned PDF, plain text and Word (42 of the 144 are PDFs). The corpus totals about 5,220 pages, with a median of 16 pages per contract and a maximum of 428.

4.2 Field catalogue

Each contract type has its own field list drawn from a shared catalogue in which a field key means the same thing in every type. Fields are typed (string, free text, list, boolean, number, date, ISO 8601 duration, money, relative date, jurisdiction, party references, structured objects and lists of objects) and prioritised: P1 for direct financial, time or exit exposure, P2 for material risk and P3 for informational fields. Roughly two thirds of the labelled values are structured (objects or lists), which matters for scoring (Section 5.2).

4.3 Labels

The 2,928 field labels were drafted by an LLM (Claude) from the full contract text and are used as they are; there is no hand-verified gold set. 1,225 labels are empty, meaning the field is not present in the contract, and 1,705 carry a verbatim evidence quote. Each label has a confidence: 2,068 high, 786 medium and 74 low.

Because the labels are drafts, we report headline results on all labels and check that each claimed effect also holds on high-confidence labels (Section 8 discusses the remaining risk).

4.4 Development and test split

Contracts were split once, stratified by contract type and source format (seed 2026, development share 0.28), into a **development set of 36 contracts** and a **test set of 108 contracts**. All tuning and every result in this draft use the development set only; the test set will be run once, at the end, with the final configurations. On the development set each run evaluates 452 labelled value cells and 280 absent-field cells per repeat.

5 Methodology

5.1 Experimental design

Each experiment compares *arms*, pipelines that differ from a reference in one setting. All arms of an experiment run on the same documents and fields, so every comparison is paired at the level of a (document, field) cell. Each arm is repeated three times at a low non-zero temperature, which lets us separate run-to-run noise from document-to-document variation. Retrieval-only experiments run the retriever for every labelled field and score the retrieved chunks without calling an LLM, which makes retrieval sweeps nearly free.

5.2 Metrics

Retrieval. For labels with an evidence quote we compute:

- **Evidence recall:** the share of labelled fields for which at least one fragment of the quote is found in a retrieved chunk. Text is normalised to word tokens, and a fragment counts as found when at least 80% of its tokens appear in order in the chunk (fragments under three tokens are ignored).
- **Recall@k** and **mean reciprocal rank (MRR)** of the first chunk holding the evidence.
- **Not indexed:** the share of quotes found in no chunk of the document at all, which isolates losses in decomposition and chunking from losses in retrieval.

Extraction. For labelled values we report four scores of increasing leniency, and two rates on absent fields:

- **Exact:** the value equals the label after type normalisation.
- **Score (partial credit):** structure-aware; object keys are averaged and list items greedily matched, with exact comparison at the leaves.
- **Lenient:** the same structure with lenient leaves (numbers by value, text by token F1).
- **Semantic:** leaves settled deterministically where possible (empty answers wrong, booleans and numbers compared, lenient matches of 1 correct); the rest graded *correct* (1), *partial* (0.5) or *wrong* (0) by an LLM grader (GPT-6 Luna) given the field definition, the label and the answer.
- **Missed:** an empty answer for a field that has a value; **absent correct** and **invented:** on fields the contract does not contain, the share answered empty versus with an invented value.

We also record LLM calls, tokens, cost (OpenRouter list prices) and wall-clock seconds per run, where one run processes one contract.

5.3 Statistical analysis

Cells from one contract are correlated, so the unit of resampling is the document. For each paired comparison we report the mean difference with a 95% confidence interval from a cluster bootstrap over documents, a permutation p-value, and McNemar's exact test for exact match, with Holm's correction within each research question. We claim an effect only when (i) its confidence interval excludes zero, (ii) it exceeds the run-to-run noise floor measured in E1, and (iii) it holds on high-confidence labels.

5.4 Models

Table 3. Models used so far.

Role	Model	Access	Price (USD per M tokens, in / out)
Reader, current system	Llama 3.1 8B Instruct	OpenRouter	0.05 / 0.08
Reader, research workhorse	Qwen3 30B-A3B Instruct 2507	OpenRouter	0.09 / 0.30
Judge, current system	Llama 3.1 8B Instruct	OpenRouter	0.05 / 0.08
Semantic grader	GPT-6 Luna (low reasoning effort)	OpenRouter	0.10 / 0.50
Grader cross-check	Gemini 4 Argon	OpenRouter	2.00 / 10.00
Embeddings	nomic-embed-text	Ollama (local)	free
Reranker	bge-reranker-v2-m3 (int8)	fastembed (local CPU)	free

6 Results

All results in this section are on the development split (36 contracts, three repeats per arm). Every number in the tables and figures is computed from the experiment exports by make_figures.py; intervals are 95% cluster bootstrap intervals over documents, and an asterisk marks a difference whose Holm-adjusted permutation p-value is below 0.05.

6.1 E1: Baseline and noise floor (RQ1)

E1 compares the current system (seven retrieval configurations, Llama 3.1 8B reader and judge) with the research baseline (R-base, Qwen3 30B, no judge). Table 4 gives the results.

Table 4. E1 on the development split (452 value cells and 280 absent cells per repeat, 3 repeats).

Metric	Current system (Llama 3.1 8B, RAG01–07 + judge)	R-base (Qwen3 30B)
Exact	0.104	0.178
Score (partial credit)	0.165	0.269
Lenient	0.222	0.328
Semantic	0.309	0.424
Missed	26.4%	28.6%
Absent correct	53.1%	76.2%
Invented on absent	45.8%	23.3%
Evidence recall	56.2%	57.1%
Score with evidence retrieved / without	0.238 / 0.071	0.395 / 0.101
LLM calls per contract	142	20
Tokens per contract	179,674	37,062
Cost per contract (USD)	0.0092	0.0035
Seconds per contract	805	242

The research baseline is better on every quality metric: the score rises by +0.104 [+0.069, +0.138] and the semantic score by +0.116 [+0.072, +0.161], and both differences hold on high-confidence labels (score +0.124 [+0.082, +0.165]). The largest change is on absent fields, where invented values fall from 46% to 23% (−22.5 [−28.4, −16.1] points). The smaller model also copies prompt placeholders into its answers ("John Doe", "ABC Corp") in 1.8% of values, against none for Qwen3. The baseline is also cheaper and faster, because the current system makes seven extraction calls and a judge call per field.

Retrieval gates reading. Within each arm, a value whose evidence was retrieved scores about four times higher than one whose evidence was not (0.395 vs 0.101 for Qwen3). Since only 57% of fields receive their evidence, improving retrieval is the first lever.

Noise floor. The standard deviation of the set mean across the three repeats is about 0.004 (0.005 for Llama, 0.003 for Qwen3), an order of magnitude below the document-level confidence interval of about ±0.035. With 36 development contracts, the number of documents, not sampling noise, limits the effects we can claim; differences smaller than roughly 0.03 in score are not resolvable on this split.

Reading given evidence. Restricting to cells whose evidence was retrieved, Qwen3 scores 0.79 on text, 0.83 on dates and 1.0 on booleans, but only 0.24 on structured (JSON) values, which make up two thirds of the labels. Inspection showed that this is mostly a scoring artefact: labels paraphrase (for example, a list of termination triggers) while the model quotes the contract. This motivated the semantic score (Section 6.9).

6.2 E11: Ensemble with judge selection (RQ5)

Because the current system stores the outcome of every retrieval configuration, the E1 runs also answer whether the ensemble helps. Selecting the judge's preferred value across seven configurations gives a score of 0.165, while RAG07 alone scores 0.175 (difference not significant, $p = 0.28$). The ensemble is therefore no better than its best member at seven times the extraction cost. The judge is the reason: of the values it passed, only 16% were correct, and it passed only 34% of the values that were correct. RAG04–RAG06 contribute almost nothing, for the reason shown in E2.

6.3 E2: Retrieval settings (RQ2)

E2 runs the retriever alone for every labelled field with an evidence quote (452 fields, no LLM calls). Table 5 shows the seeded configurations.

Table 5. Evidence recall of the seeded retrieval configurations (development split, 452 fields).

Config	Settings	Recall	Recall@1	MRR	Mean chunks
RAG01	dense, top 10, threshold 0.5, expansion	50.0%	23.7%	0.340	3.8
RAG02	dense, top 5, threshold 0.5, expansion	49.8%	23.7%	0.340	3.7
RAG03	dense, top 3, threshold 0.5, expansion	43.4%	23.7%	0.325	2.8
RAG04–06	dense, top 10 / 5 / 3, threshold 0.7, no expansion	2.2%	2.2%	0.022	0.05
RAG07	RAG02 + BM25 (RRF) + 2 anchor chunks	71.2%	37.8%	0.494	5.2

Three findings shape the rest of the programme:

- **Absolute thresholds starve the reader.** nomic-embed-text cosine similarities for relevant chunks mostly lie between 0.4 and 0.7, so a 0.7 threshold returns on average 0.05 chunks per field and recall collapses to 2.2%. The extraction model then answers from no evidence at all. The research baseline R-base therefore uses a fixed *k* with no threshold.
- **top-k is capped by hits per query.** With three hits per query, raising top-k from 5 to 20 changes nothing, because the merged pool never holds more than about five unique chunks. Recall only grows with *k* when the per-query pool is raised as well (20 hits per query).
- **Hybrid search with anchors beats dense search at every k,** by 15.7 points at k = 5 (72.8% vs 57.1%) and 6.2 points at k = 20 (91.4% vs 85.2%), with a much better first hit (recall@1 37% vs 25%). Exact terms, defined names and dates favour lexical matching, and cover- and signature-page fields favour anchors.

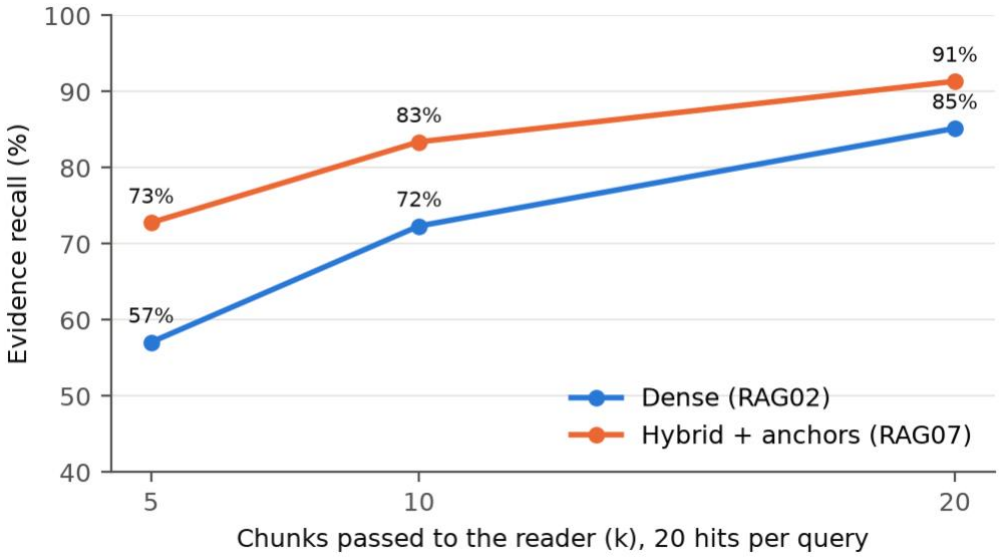


Figure 3. Evidence recall against the number of chunks passed to the reader, for dense and hybrid retrieval (20 hits per query, no threshold, development split). Recall keeps rising to k = 20, so ranking, not the candidate pool, limits recall at k = 5.

Only 2.0% of evidence quotes are missing from every chunk of their document, so decomposition and chunking lose little; the evidence is indexed but ranked too low. With RAG07, recall at k = 5 plateaus at about 70% while k = 20 reaches 91%. This pointed to two remedies, tested in E6: rank better (a reranker) or pass more chunks (larger k). Note that k = 20 (about 9,000 prompt tokens) exceeds the 8,192-token context window configured for local Llama 3.1 8B, so the second remedy is open only to readers with longer contexts.

6.4 E6: Cross-encoder reranking, retrieval only (RQ3)

Table 6. Evidence recall at k = 5 with and without the reranker (20 hits per query, development split, 452 fields).

Retrieval	Candidates	Recall	Recall@1	MRR	Δ recall vs no reranker (points, 95% CI)
R-base (dense)	—	57.1%	24.6%	0.365	
R-base + reranker	20	68.1%	37.8%	0.494	+11.1 [+5.9, +16.0]*
R-base + reranker	40	70.1%	37.2%	0.496	+13.1 [+7.5, +18.5]*
RAG07 (hybrid + anchors)	—	70.8%	37.2%	0.489	
RAG07 + reranker	20	74.3%	40.5%	0.526	+3.5 [−1.1, +8.1] (n.s.)
RAG07 + reranker	40	75.0%	39.8%	0.524	+4.2 [−0.2, +8.5] (n.s.)

The reranker lifts dense retrieval by 13.1 points, a significant gain that brings it to the level of hybrid retrieval without a reranker. On hybrid retrieval the gain is 4.2 points and not significant (95% CI −0.2 to +8.5 points). The reranker therefore *substitutes* for BM25 and anchors rather than adding to them, and at k = 5 it still falls far short of the recall that k = 20 reaches without any reranking. On CPU the int8 model costs about 0.1 s per candidate; running it in eight parallel workers exhausted the memory of a 16 GB laptop, which we fixed with an on-disk score cache, lazy model loading and a thread cap.

6.5 E6: Context size versus reranking, end to end (RQ3, RQ4)

E6 then measures whether the recall gains reach the extracted values. All four arms use hybrid retrieval with anchors and 20 hits per query (R07) and the Qwen3 30B reader; they differ only in how many chunks reach the reader (5, 10 or 20) and whether the reranker selects the five (Table 7, Figure 4). Figure 5 gives the paired differences against k = 5.

Table 7. End-to-end results by context size and reranking (Qwen3 30B, development split, 3 repeats). Best value per row in bold.

Metric	R07, k = 5	R07, k = 10	R07, k = 20	R07, k = 5 + reranker
Evidence recall	70.8%	82.1%	88.7%	73.8%
Exact	0.215	0.252	0.266	0.232
Score (partial credit)	0.332	0.385	0.406	0.346
Lenient	0.393	0.453	0.477	0.405
Semantic	0.498	0.565	0.594	0.516
Missed	22.6%	17.1%	14.0%	20.9%
Absent correct	74.2%	71.8%	69.3%	75.8%
Invented on absent	25.8%	28.1%	30.0%	23.7%
Tokens per contract	40,850	68,245	109,309	45,356
Cost per contract (USD)	0.0039	0.0064	0.0101	0.0043
Failed runs (of 108 + retries)	1	1	2	6

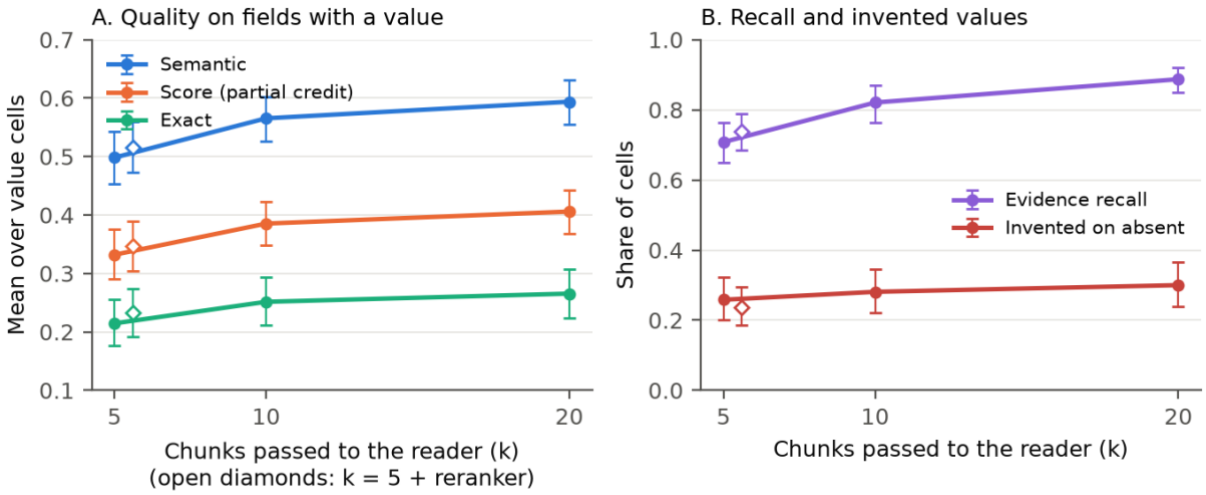


Figure 4. Quality, evidence recall and invented values against the number of chunks passed to the reader (Qwen3 30B, development split, 95% cluster bootstrap intervals). Open diamonds: five chunks chosen by the reranker.

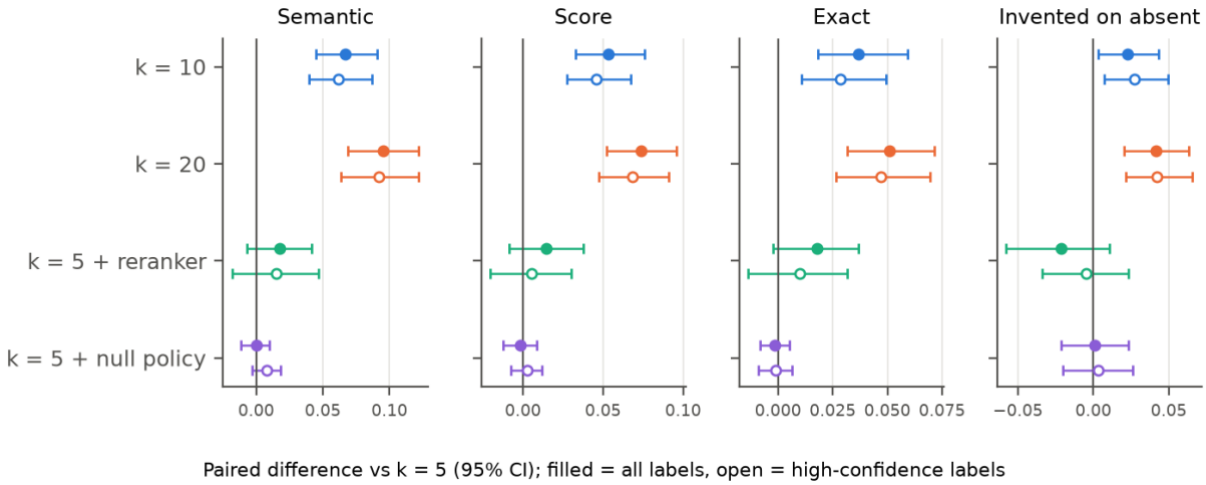


Figure 5. Paired differences against k = 5 for every arm tested on the same cells, including the null-policy prompt of E8a (Section 6.8). Filled markers: all labels; open markers: high-confidence labels only.

More context wins, with diminishing returns. Against k = 5, passing 10 chunks raises the semantic score by +0.067 [+0.045, +0.091] and passing 20 by +0.096 [+0.069, +0.122]; k = 20 is still better than k = 10 (+0.028

[+0.009, +0.047]). Missed values fall from 22.6% to 14.0%. All of these differences are significant and hold on high-confidence labels. The reader is not overwhelmed by four times as many chunks; the recall gain carries through to the answers.

At a price in hallucination. With more context the model also fills more fields that the contract does not contain: invented values on absent fields rise from 25.8% to 28.1% at k = 10 (+2.3 [+0.3, +4.3] points, not significant after Holm) and 30.0% at k = 20 (+4.2 [+2.1, +6.3] points, significant).

Cost-effectiveness. Tokens and cost rise with k, to 0.0064 and 0.0101 USD per contract. k = 10 obtains 70% of k = 20's semantic gain for 40% of its extra cost, so it is the better default when cost matters and k = 20 when accuracy does (Figure 6). Both lie on the cost-quality Pareto front, together with the cheaper R-base and k = 5 arms.

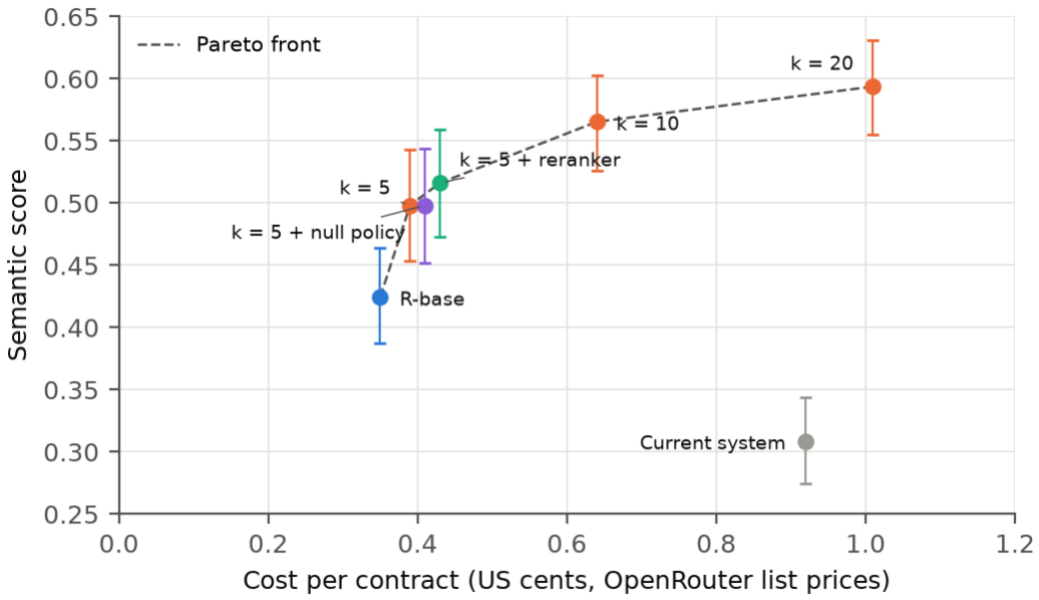


Figure 6. Cost per contract against semantic score for every arm run so far (95% intervals). Arms from different experiments ran on different code versions, so the comparison across experiments is indicative; the dashed line joins the arms no cheaper arm beats.

Reranking does not reach the answers. The reranker arm moves every metric in the right direction (semantic +0.018, score +0.014, invented −2.1 [−5.8, +1.1] points), but none of these differences is significant. For this reader, recall gained by passing more chunks is worth far more than recall gained by ranking them better.

As an indication only, the k = 5 hybrid arm (score 0.332) is well above the dense R-base arm of E1 (0.269) with the same reader; the two runs come from different code versions and are not yet compared formally. The clean dense-versus-hybrid comparison belongs to E5.

6.6 Retrieval failures versus reading failures

Figure 7 splits every labelled value into four outcomes by its semantic grade: correct, partially correct, wrong although the evidence was retrieved (a reading failure), and wrong because the evidence was not retrieved (a retrieval failure). This attributes the remaining error to a stage of the pipeline.

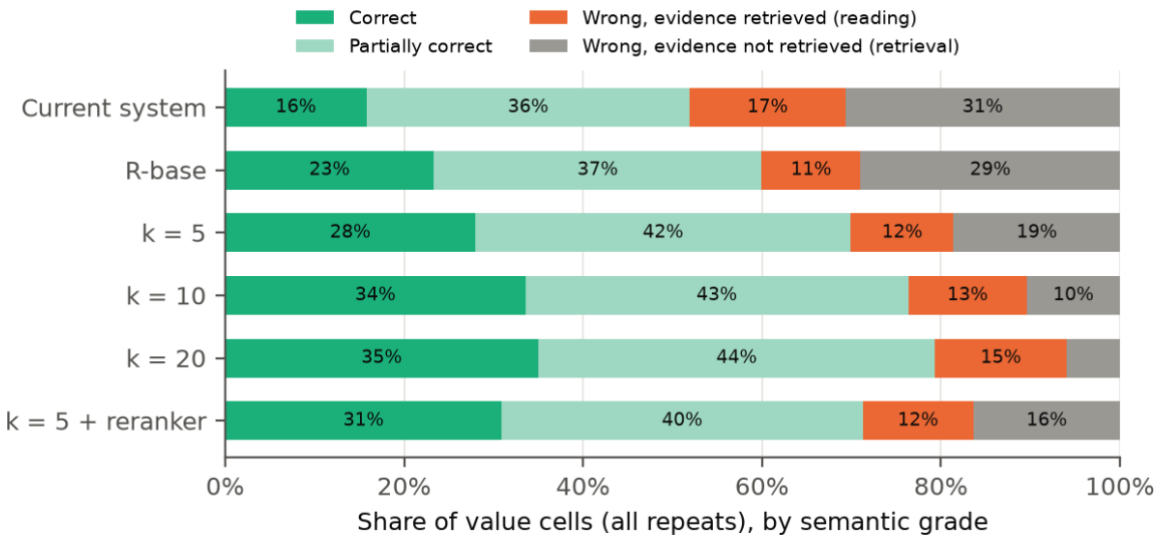


Figure 7. Outcome of every labelled value (all repeats) by arm. Grey: wrong and evidence not retrieved; orange: wrong although the evidence was retrieved.

Retrieval failures fall from 31% of values in the current system and 29% under R-base to 19% at k = 5, 10% at k = 10 and 6% at k = 20. Reading failures stay between 11% and 15% for Qwen3 (17% for Llama) and grow slightly with k. At k = 20 the reader, not the retriever, accounts for most wrong values, and the largest block is now partially correct values (44%), mostly structured fields with some sub-fields missing. Further gains must therefore come from the reader (E8) and from the scoring of structured values, not from more retrieval.

6.7 Where invented values come from

Invented values on absent fields are concentrated in a few fields: the eight fields in Table 8 account for 67% of all invented values at k = 5, and the first four for 48%. Inspecting the answers shows that most are not hallucinations in the usual sense (Table 8):

Table 8. Invented-value rate on absent labels for the fields with the most invented values (development split, all repeats).

Field	Absent cells per repeat	k = 5	k = 20	k = 5 + null policy	What the model returns
expiration_date	15	98%	100%	93%	an expiry relative to an event (e.g. 90 days after termination) where the label is empty
uncapped_liabilities	10	70%	83%	87%	the definition's example list (fraud, IP, data breach), copied verbatim
initial_term	18	37%	41%	39%	a duration derived from dates the contract gives
document_status	7	90%	95%	95%	"executed"; every such label has low or medium confidence
execution_date	16	29%	50%	38%	dates from elsewhere in the text, some placeholders (2026-01-01)
effective_date	7	57%	62%	52%	not yet inspected
renewal	22	15%	14%	14%	not yet inspected
non_compete	10	20%	37%	23%	not yet inspected

- **Label policy, not invention.** For expiration_date the model returns an expiry relative to an event (for example 90 days after termination) where the label is empty; the field's own format help asks for exactly this. For document_status it answers "executed", and every one of those labels has low or medium confidence. These cells measure a disagreement about what the field should hold.

- **Copying the field definition.** uncapped_liabilities is defined as "Liabilities carved out of the cap (fraud, IP, data breach)", and the model returns exactly ["fraud", "IP", "data breach"] in 74 absent cells and 53 value cells across the E6 and E8a arms. In 8 of the 10 contracts where this happens, the retrieved chunks mention neither fraud nor a data breach.
- **Derived and placeholder values.** initial_term is computed from dates the contract gives, and a few execution dates are placeholders such as 2026-01-01.

6.8 E8a: An explicit null policy in the system prompt

E8a tests the obvious fix: a system prompt that adds three rules to the original (return null unless the text explicitly states the value; examples in a field definition and the JSON template are never values to copy; blanks, template wording and sample data are not values). Everything else equals the k = 5 arm, so the comparison is paired on the same cells (Table 9).

Table 9. E8a: null-policy prompt against the original prompt (R07, k = 5, Qwen3 30B, development split, 3 repeats).

Metric	Original prompt	Null policy	Difference (95% CI)	p
Semantic	0.498	0.498	−0.000 [−0.011, +0.010]	1.000
Score (partial credit)	0.332	0.331	−0.001 [−0.012, +0.009]	0.795
Exact	21.5%	21.3%	−0.1 [−0.8, +0.5] points	0.831
Invented on absent	25.8%	26.0%	+0.1 [−2.1, +2.3] points	1.000
Invented on absent, high confidence	18.6%	18.9%	+0.3 [−2.0, +2.6] points	0.907
Cost per contract (USD)	0.0039	0.0041		

The prompt had no measurable effect on any metric, and the copied definition example became, if anything, more frequent (12 to 18 absent cells). The logged LLM calls confirm that every E8a call carried the new rules. The generic field prompt already told the model not to infer, guess or invent values, so the model had been ignoring that instruction too. The lesson for practice is to remove errors at their source, in the field definitions and the labelling policy, rather than adding instructions the reader does not follow.

6.9 Validating the semantic score (RQ6)

Exact and partial-credit scores count paraphrased but correct answers as wrong, which is most visible on structured fields. Under the semantic score, Qwen3's score on structured values whose evidence was retrieved rises from 0.24 to 0.47, and on lists from 0.10 to 0.60, while the ranking of systems is unchanged (Tables 4 and 7). Grading all E1 runs cost USD 0.12.

The grader was not validated against human grades. Instead, 150 randomly sampled cells from E1 and E6 were re-graded by a second, independent model from another provider (Gemini 4 Argon; Table 10). The two graders agree on 82% of cells (Cohen's kappa 0.68, substantial agreement) and on 89% for the binary decision correct versus not correct. The main disagreement is partial versus correct (15 of 150 cells), with GPT-6 Luna slightly stricter; only 2 cells were graded correct by one model and wrong by the other. We therefore treat semantic scores as reliable for comparing arms, which share the grader, but not as calibrated absolute accuracies.

Table 10. Grades of the two graders on 150 sampled cells (rows: GPT-6 Luna, columns: Gemini 4 Argon).

GPT-6 Luna \ Gemini 4 Argon	correct	partial	wrong	Total
correct	20	0	1	21
partial	15	77	7	99
wrong	1	3	26	30

7 Discussion

Fix retrieval first, then the reader. Almost all evidence is indexed, so the losses were ranking losses: every change that raised recall also raised extraction quality, while chunking cost only 2% of evidence. Once 20 chunks reach the reader, retrieval failures fall to a few per cent and wrong or partially correct readings dominate, so the next gains must come from the reader and from how structured values are scored.

Check retrieval on its own. The context-starving threshold and the silently capped top-k of the current system are invisible in an end-to-end figure but obvious in a retrieval-only evaluation that costs nothing. Likewise, an ensemble judged by a small model only adds cost: a judge helps only if it verifies better than the reader extracts, which E9 will test against a deterministic evidence check.

More context beats better ranking, for a long-context reader. The "lost in the middle" effect [10] does not outweigh the recall gain at about 9,000 tokens per call. A reranker matters mainly when the reader can take only a few chunks, as with an 8k-context local model. The price of more context is more values for absent fields, and these trace to field definitions and labelling policy rather than to instructions the model can be told to follow.

State the notion of correctness. Under exact scoring structured fields looked badly read; under semantic scoring much of that gap was paraphrase. Strict and lenient numbers should not be compared across studies.

8 Limitations and Improvements

8.1 Threats to validity

- **Draft labels.** Labels were written by an LLM and not checked by hand, and some absent labels record a policy choice (relative expiry dates, document status), so the invented-value rate overstates hallucination on those fields. Label style may favour Claude-family models, so the planned ceiling model comes from another family; every claim is checked on high-confidence labels.
- **LLM-graded semantic score.** It agrees substantially with a second grader (kappa 0.68) but is not human-validated; exact, partial-credit and lenient scores are reported alongside it.
- **Small development split, also used for tuning.** With 36 contracts, effects below about 0.03 are not resolvable and reported gains are optimistic; the final claims rest on the 108-contract test split, run once.
- **One reader and changing code.** All grids use Qwen3 30B, and the experiments span several commits. Arms are compared directly only on identical extraction code (E6, E6b, E8a); the baseline will be re-run on the frozen final code.
- **Public documents and fuzzy evidence matching.** The SEC filings may be in the models' pre-training data, and evidence recall relies on 80% token coverage of a quote, so absent fields and multi-passage values are not covered.

8.2 Improvements

Building the platform and running the first experiments exposed the following weaknesses, ordered by priority (Table 11).

Table 11. Weaknesses found and the improvement each suggests.

Priority	Weakness	Improvement
High	The default pipeline is the seven-configuration ensemble with a judge, the weakest and costliest setup measured	make hybrid retrieval with anchors, k = 10, no judge the default
High	Labels are LLM drafts with no policy for derived or relative values	hand-verify 30–50 contracts; write a labelling policy
High	The reader ignores prompt rules (E8a)	schema-constrained output and a required evidence span checked against the document (E8, E9)
Medium	No schema migrations: the reranker columns were added to the live database by hand	versioned migrations (e.g. Alembic)
Medium	No continuous integration; tests run only when a developer runs them	CI for the three offline suites, type checks and linter
Medium	The reranker settings exist only in the schema and SQL presets; a failed provider call fails the whole run	expose every pipeline flag in the API and UI; retry single fields with back-off
Medium	Structured results are shown as raw JSON side by side (Figure 2); about 44% of values are partially correct	a field-by-field diff view; report sub-field recall and precision
Low	Eight workers each loading the reranker exhausted a 16 GB machine	one shared model service, or workers that declare the resources a run needs
Low	No authentication; progress is polled every two seconds	authentication in the gateway; server-sent events

9 Conclusion and Next Steps

On the development split, the largest gains came not from a bigger ensemble or a judge but from getting the right text to the reader: removing a context-starving threshold, adding lexical search and anchor chunks, and passing more chunks to a reader that can use them. A single hybrid configuration with 10 chunks and Qwen3 30B reaches a semantic score of 0.57 for 0.0064 USD per contract (0.59 with 20 chunks), against 0.31 for the current seven-configuration system. The reranker's retrieval gain does not reach the answers, and the extra invented values that come with more context call for better field definitions and labels, not stronger instructions.

Table 12 gives the status of the programme. LLM spending so far is about USD 5.05, measured from the logged calls at OpenRouter list prices (including USD 0.53 for semantic grading and the grader cross-check). The remaining experiments run in the order shown, on the development split, with retrieval variants scored first by the free retrieval-only evaluation; fine-tuning follows once the best approach is fixed, and an error analysis of 50 development-split failures decides whether it targets the retriever or the reader.

Table 12. Experiment programme: status, cost so far and estimated cost of the remaining steps (USD).

#	Question	Status	Cost
E1	Baseline and noise floor	Done (6.1)	1.37
E2	Retrieval sweep: top-k, threshold, hits per query	Done (6.3)	0
E6	Reranking and context size (k = 5, 10, 20)	Done (6.4–6.6)	2.71
E8	Reader: null-policy prompt	Done (6.8)	0.44
E11	Ensemble with judge selection	Done (6.2)	0 (reuses E1)
E5	Query expansion × hybrid search; HyDE and description queries	Next, step 1	≈ 1.5
E3	Embedding models, nomic task prefixes	Step 2	≈ 1.5
E4	Structure-aware and contextual chunking	Step 3	≈ 1.5 (+ 4–6)
E8	Reader: group calls, constrained output, quote-then-answer, oracle evidence	Step 4	≈ 3
E9	Verification: LLM judge vs evidence-span check	Step 5	≈ 2
E7	PDF decomposition method	Step 6	≈ 1.5
E12	Leave-one-out ablation; transfer to a local model	Step 7	≈ 3
E10	RAG vs the whole contract in context	Step 8	≈ 1–6
FT	Fine-tune Llama 3.1 8B (LoRA) on the best pipeline's retrieved context	Step 9	local, no API cost
E13	Test split, once: baseline, full stack, fine-tuned model, long context, frontier ceiling	Step 10	≈ 30

References

[1] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap.* Chapman & Hall, 1993.

[2] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian and Z. Liu. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *Findings of ACL*, 2024.

[3] G. V. Cormack, C. L. A. Clarke and S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. *SIGIR*, 2009.

[4] S. Es, J. James, L. Espinosa-Anke and S. Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. *EACL System Demonstrations*, 2024.

[5] R. Dror, G. Baumer, S. Shlomov and R. Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. *ACL*, 2018.

[6] D. Hendrycks, C. Burns, A. Chen and S. Ball. CUAD: An expert-annotated NLP dataset for legal contract review. *NeurIPS Datasets and Benchmarks*, 2021.

[7] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[8] Y. Koreeda and C. D. Manning. ContractNLI: A dataset for document-level natural language inference for contracts. *Findings of EMNLP*, 2021.

[9] P. Lewis, E. Perez, A. Piktus et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020.

[10] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni and P. Liang. Lost in the middle: How language models use long contexts. *TACL*, 12, 2024.

[11] R. Nogueira and K. Cho. Passage re-ranking with BERT. arXiv:1901.04085, 2019.

[12] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.

[13] J. Saad-Falcon, O. Khattab, C. Potts and M. Zaharia. ARES: An automated evaluation framework for retrieval-augmented generation systems. *NAACL*, 2024.

[14] D. Tuggener, P. von Däniken, T. Peetz and M. Cieliebak. LEDGAR: A large-scale multi-label corpus for text classification of legal provisions in contracts. *LREC*, 2020.

[15] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[16] L. Zheng, W.-L. Chiang, Y. Sheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*, 2023.